

Discrimination Sheet — A vs B

Every "which one?" decision in the course. The exam asks these, not definitions.

How to use this: cover the right-hand side. Read the options, decide what would make you pick each one, *then* check. The orange line under each table is the **scenario signal** — the phrase in a question that points to the answer.

1. Improving the Model

RAG vs Fine-tuning vs Prompt engineering

Option	Choose when
RAG	You need facts — especially ones that change, need citations, or need per-tenant filtering
Fine-tuning	You need behaviour — style, format, domain language. Does not fix the knowledge cutoff.
Prompt engineering	You need output control — structure, tone, constraints, reasoning steps

Signal: "up-to-date information", "cite sources" → RAG. "Learn our terminology / output format" → fine-tuning. "Consistent structure / required disclaimer" → prompting. Real systems use fine-tuning for format *and* RAG for facts.

Full fine-tuning vs LoRA

Option	Choose when
Full fine-tuning	Maximum accuracy, budget and GPU time available
LoRA	Base model frozen, small adapter trained (~1%). One base, many adapters. ~90% of the benefit at ~5% of the cost.

Signal: "reduce computational requirements", "multiple specialities on one base model" → LoRA.

Flows vs Agents

Option	Who decides the sequence
Prompt Flows	You — graph defined in advance. Deterministic, auditable.
Agents	The model — picks its own tools. Flexible, non-deterministic, harder to audit.

Signal: regulated industry, "auditable", "same path every time" → Flows. "Multi-step problems we can't anticipate" → Agents.

Single agent vs Multi-agent

Option	Choose when
Single	One domain, few tools, sequential work
Multi-agent (Strands + Agent Squad)	Distinct expertise required, work parallelisable, each part complex on its own

Signal: different specialisms named (forecasting, fraud, logistics) → multi-agent. Bedrock Agents = one managed agent; **Strands** = code-first custom agents; **Agent Squad** = orchestration *between* agents.

2. Model Selection & Routing

Tiered routing vs Cascading

Option	Mechanism and cost
Tiered routing	Assess complexity <i>first</i> , then call the right model — one call , cheaper
Cascading	Try the small model, escalate on low confidence — two calls when it escalates, but based on real confidence

Signal: "confidence threshold" → cascading. "Classify query complexity" → tiered. Cascading only pays off when most queries genuinely are simple.

Ensemble aggregation strategies

Strategy	Task type
Majority voting	Classification, discrete choices
Weighted averaging	Numeric predictions; weights from historical performance
Ranked fusion	Retrieval — merging several ranked lists

Which hosting option

Option	Choose when
Bedrock On-Demand	Uneven traffic, no commitment; shared capacity means possible throttling
Bedrock Provisioned	Steady high volume; Model Units; no throttling; term commitment
Bedrock Batch	Not real-time — ~50% cheaper
Custom Model Import	Your fine-tuned model, Bedrock serverless pricing — no hourly endpoint charge
SageMaker real-time	Own model, always on, low latency
SageMaker serverless	Own model, intermittent traffic, cold starts acceptable
SageMaker async	Large payloads, long inference, scale to zero

Signal: "custom model but avoid provisioned throughput cost" → Custom Model Import. "Results tomorrow morning is fine" → Batch or async.

3. Retrieval & Vector Stores

Which vector store

Option	Choose when
Bedrock Knowledge Bases	Fully managed, standard RAG, nothing custom needed
OpenSearch Serverless	Hybrid search , fine-grained security, binary vectors/FP16, tunable indices
Aurora + pgvector	SQL filter and vector similarity in one query ; already on Postgres
S3 Vectors	Very large corpus, cost-optimised, sub-second acceptable
DynamoDB + Faiss/Pinecone	Sub-ms simple lookups (DynamoDB does not do vector search alone)
Neptune Analytics	Graph + vector — "similar to this AND cited by that"

Hard constraint: SharePoint, Confluence or Salesforce data sources → **only OpenSearch Serverless**. That single fact rules out S3 Vectors in several scenarios.

Semantic vs Hybrid search

Option	Strength / weakness
Semantic (vectors only)	Catches paraphrase and cross-language. Fails on exact terms — "Section 230" also returns Section 220.
Hybrid (vectors + BM25)	Both. Configured via <code>OverrideSearchType = HYBRID</code> , and requires OpenSearch Serverless with filterable text fields .

Signal: citations, statute numbers, product codes, proper nouns → hybrid.

ANN algorithms

Algorithm	Trade-off
HNSW	Graph-based; fast + high recall; memory-hungry
IVF	Cluster-based; lower memory; good at very large scale
LSH	Hash-based; fast, less accurate
PQ	Compression; much less memory, some accuracy loss

HNSW parameter	Affects
<code>m</code>	Connections per node → recall, memory , build time
<code>ef_construction</code>	Index quality → build time (not search speed)
<code>ef_search</code>	Recall → query time

There is no universal best — **benchmark on your own dataset**. If the question mentions memory pressure at scale, IVF or PQ.

Chunking strategies

Strategy	Choose when
Fixed-size	Plain narrative text with no structure
Structure-aware	Documents with headings/sections — convert to HTML first, cut on tags
Hierarchical (parent-child)	Search the small chunk, send the parent — precision plus context
Semantic	Cut where meaning shifts; expensive (embeddings during chunking)

Reranking vs bigger k

Option	Why
Reranker model	Reads query and document together , so far more accurate — but too slow to run over the whole corpus. Two stages: top 100 → top 10.

4. Data Quality & Preparation

Glue Data Quality vs SageMaker Data Wrangler

Tool	Catches
Glue Data Quality	Known unknowns — DQDL rules you wrote. Automated, production, recurring.
Data Wrangler	Unknown unknowns — visual exploration surfaces problems you didn't know to check for. Exploration, not a pipeline.

Glue vs AppFlow

Tool	Source type
AWS Glue	S3, databases, big ETL. Job bookmarks give incremental processing.
Amazon AppFlow	SaaS applications — pre-built connectors, bidirectional, filtered

Comprehend vs Macie

Service	Operates on
Comprehend	Text, in real time — 25+ PII types at request time. Comprehend Medical for PHI.
Macie	Data at rest in S3 — discovers sensitive data you didn't know was there

Rekognition vs SageMaker Processing

Service	Nature
Rekognition	Generic managed vision — objects, quality, text in image
SageMaker Processing	Custom — damage severity scoring, domain-specific analysis at scale

5. Integration & Messaging

SQS vs SNS vs EventBridge vs Kinesis vs MQ

Service	Choose when
SQS	Queue, one consumer group, buffering, rate limiting
SQS FIFO	Ordering required — message group ID gives order within a group and parallelism across groups
SNS	Simple fan-out, no filtering
EventBridge	Content-based routing , many targets, SaaS integrations, schema registry, input transformers
Kinesis	Ordered streaming with replay
Amazon MQ	Legacy already speaks JMS/AMQP/MQTT — nothing else justifies it

Signal: "route only events where amount > X" → EventBridge. "Existing message broker" → MQ. "Smooth out spikes / avoid throttling" → SQS.

AppSync vs API Gateway vs Lambda Function URL

Option	Choose when
AppSync	GraphQL, subscriptions for streaming, field-level auth, multiple client shapes, offline sync
API Gateway	REST, usage plans, API keys, per-tier throttling, existing REST estate
Lambda Function URL	Response streaming — API Gateway cannot stream, it buffers

Timeouts are **not** the differentiator — both AppSync and API Gateway are ~30s. Streaming and auth model are.

WebSocket vs SSE vs Chunked encoding

Option	Characteristics
WebSocket	Bidirectional, persistent; ping every 30–60s; you build resume yourself
SSE	Server → client only, plain HTTP, resume built in via event IDs — usually enough for LLM streaming
Chunked encoding	Simplest plain-HTTP streaming, 1–4 KB chunks

Signal: "recover after a dropped connection without regenerating" → SSE. "Client also sends messages" → WebSocket.

Action Groups vs MCP

Option	Scope
Action groups	Bedrock Agents only; OpenAPI schema + Lambda
MCP	Standard protocol — any MCP-compatible client. Build once, reuse across models.
MCP server host	Choose when
Lambda	Stateless, lightweight, under 15 minutes — calculations, lookups, search
ECS	Heavy, stateful, specialised environment — simulation engines, code execution

6. Security & Access

IAM vs SCP vs Verified Permissions vs Lake Formation

Control	Answers
IAM	Can you call Bedrock at all? Grants permission.
SCP	Organisation-wide ceiling . Never grants — only limits. Account admins cannot bypass.
Permission boundary	Ceiling on what a <i>delegated admin</i> may grant
Verified Permissions (Cedar)	Can you use it on this specific document ? Classification, department, business context.
Lake Formation	Inside a table — column, row, cell level, where IAM only reaches the bucket

Signal: "even an account administrator must not be able to..." → SCP. "Doctors see diagnosis but not SSN" → Lake Formation column-level.

IAM Identity Center vs Cognito

Service	For
IAM Identity Center	Workforce — employees, internal applications
Cognito	Customers — external users. Pre-token Lambda injects enterprise claims.

Interface vs Gateway VPC endpoints

Type	Applies to
Interface	Bedrock, Comprehend, most services — creates an ENI, supports security groups
Gateway	Only S3 and DynamoDB — a route table entry

Masking techniques

Technique	Reversible / context
Redaction	Not reversible; least context — the model can't tell two mentions are the same person
Tokenization	Reversible via a mapping table; consistent references preserved
Pseudonymization	Reversible; most context — reads as natural text
Format-preserving encryption	Same length and shape, so legacy systems still accept it

Signal: "recover the original later" → tokenization. "Legacy system expects the same format" → FPE. "Model needs fluent text" → pseudonymization.

Anonymization techniques

Technique	Mechanism
Generalization	Age 34 → 30–40; ZIP 90210 → 902**
Perturbation	Add noise to individual values; aggregates hold
Synthetic data	Entirely fabricated dataset with the same statistics — safest
Differential privacy	Noise on aggregate query results , with a privacy budget so repeated queries can't average it away
K-anonymity	Every record indistinguishable from at least k–1 others

7. Deployment & Release

Shadow vs Canary vs Progressive vs Blue/Green

Strategy	Users see	Risk / cost
Shadow	The old model — the new one runs on real traffic but its output is discarded	Zero risk , double cost
Canary	The new model, small share of users	Low risk
Progressive exposure	The new model, growing share (5 → 25 → 50 → 100%)	Low, decreasing
Blue/Green	The new model, everyone at once	Higher, but instant rollback; double environments

Signal: "test with production traffic without affecting users" → **shadow**. Natural order: shadow → canary → progressive → full.

A/B testing vs Canary

Technique	Purpose
A/B testing	Comparison — which performs better, with statistical significance
Canary	Risk reduction — roll out safely, not to compare

AppConfig vs Parameter Store vs Lambda alias

Mechanism	Switches
Parameter Store	Simple key-value config
AppConfig	Config values with feature flags, gradual rollout, validation, auto-rollback
Lambda alias	Code version ; weighted aliases give canary

8. Cost & Performance

Caching types

Type	Caches / when
Semantic caching	Full response matched by embedding similarity (85-95%) — users ask the same thing in different words
Result fingerprinting	Response matched by normalised query — cheaper, catches only minor variations
Prompt caching	The static prefix of the prompt; ~90% discount on cached tokens. Static content must come first.
CloudFront edge	Non-personalised responses, geographically distributed

Latency vs cost vs throughput

Want	Cost
Low latency	Provisioned capacity → expensive
Low cost	Queue + batch → higher latency
High throughput	Batching → higher per-request latency

You cannot have all three. Also: **streaming changes perceived latency, not actual latency** — if the question says "reduce processing time", streaming is the wrong answer.

Provisioned vs On-Demand — the break-even

Rule	
Choose provisioned	Only when on-demand cost <i>at your volume</i> exceeds the commitment. Otherwise you pay for idle capacity.

Model compression

Technique	Mechanism
Quantization	Lower precision — FP32 → FP16 → INT8 → INT4
Pruning	Removes low-contribution weights
Distillation	Large teacher trains a small student

9. Inference Parameters

Parameter	Controls
temperature	Randomness. 0 = deterministic. Low for extraction/classification; regulated domains use 0-0.2.
top_k	Fixed count of candidate tokens
top_p	Adaptive — enough tokens to reach probability mass p. Usually the better control.
max_tokens	Output length only. Output tokens are the expensive ones.

Rule: tune temperature *or* top_p, never both — you won't be able to attribute the effect.

10. Safety, Evaluation & Troubleshooting

Prompt injection vs Jailbreak

Attack	Goal
Prompt injection	Override the system's instructions . Indirect injection hides inside a document that RAG later retrieves — far more dangerous, since it fires for every user.
Jailbreak	Bypass the safety restrictions — role-play, hypotheticals, encoding, language switching, gradual escalation

Guardrails vs output validation Lambda

Control	Checks
Guardrails	Harmful content, denied topics, PII, contextual grounding — generic and model-independent
Output validation Lambda	Business rules — required disclaimers, fee accuracy, "never show a full account number"

The three FM troubleshooting techniques

Technique	Detects
Golden datasets	Hallucination — the answer is factually wrong
Output diffing	Inconsistency — the answer changes when it shouldn't. Variation testing (same time, different phrasing) vs temporal (same phrasing, different time).
Reasoning path tracing (CoT)	Logical error — the answer may look right but the reasoning is invalid; catches circular reasoning

Three distinct failures. **No single technique catches all three** — and the remediations differ, so triage matters: hallucination → grounding; reasoning error → CoT prompt; inconsistency → lower temperature.

RAG evaluation types

Type	Isolates
Retrieve only	Retrieval alone — did the right chunks come back?
Retrieve and generate	The whole pipeline

If retrieve-only passes but the final answer is wrong, **the fault is in the model, not the retriever**. That inference is the point of having two modes.

Citation precision vs coverage

Metric	Asks
Precision	Are the citations given correct ?
Coverage	Is everything that needed citing actually cited?

Monitoring services

Service	Answers
CloudWatch	How many? How slow? Is it running?
X-Ray	Where did the time go, across hops
CloudWatch RUM	What the user actually experienced , client-side
SageMaker Model Monitor	Are the answers still correct ? (drift)
Synthetic monitoring	Generates its own traffic — detects problems before users do

Pre-training vs post-training bias

Stage	Question
Pre-training	Is the bias in the data ?
Post-training	Is the bias in the predictions ?

Clarify metrics: **DPPL** (difference in positive proportions), **DI** (disparate impact — a ratio), **DCR** (difference in conditional rejection).

Fairness metrics

Metric	Definition
Demographic parity	Equal positive rates across groups
Equal opportunity	Equal rates among those who qualify
Equalized odds	Equal TPR <i>and</i> FPR — strictest
Counterfactual fairness	Change only the demographic attribute; the answer must not change. The most practical test for GenAI.

Retriable vs non-retriable errors

Codes	Action
429 / 500 / 503	Retry . 429 → almost immediate retry with jitter; 5xx → exponential backoff.
400 / 401 / 403	Do not retry — the request or the credentials are wrong

Circuit breaker vs retry vs fallback

Mechanism	Handles
Retry + backoff	Transient failure — one-off throttle or glitch
Circuit breaker	Sustained failure — stop trying, prevent cascade (closed / open / half-open)
Graceful degradation	Everything is down — smaller model → RAG → rule-based → human
CloudWatch custom metrics	Degraded output — running, no errors, answers are worse

The Ten Signals Most Likely to Decide a Question

Phrase in the scenario	Points to
"data cannot leave our premises"	Outposts
"without affecting users"	Shadow deployment
"even the account administrator"	SCP
"on this specific document"	Verified Permissions
"exact statute / product code"	Hybrid search
"the legacy system already uses JMS"	Amazon MQ
"custom model, avoid provisioned cost"	Bedrock Custom Model Import
"resume after a dropped connection"	SSE with event IDs
"detect it before users are affected"	Synthetic monitoring
"guaranteed accurate figures"	Text-to-SQL (answer comes from the database)